

Computer Lab 1 on Inference Theory

1 Introduction

This computer lab consists of two main parts:

1. Confidence intervals
2. Hypothesis testing

Try to answer all questions. Ask when you do not understand. You will analyse only simulated/computer generated data for this lab. In the next lab, computer lab 2 in inference theory, you will do analyse real life data.

2 Confidence intervals

This part of the computer lab consists of learning how to construct confidence intervals and to illustrate the covering probability/confidence grade of a confidence interval.

1. We want to construct a confidence interval for the expectation μ in a Normal distribution with known variance. We therefore generate a sample of size $n = 200$, from a $N(\mu, 1)$ distribution. Choose a suitable value of μ yourself. (In the code below we have chosen $\mu = 2$, you may choose another value, it is of no importance for the results of the lab).

```
n<-200
mu<-2
n<-200
x<-rnorm(n,mean=mu,sd=1)
```

The vector x now contains all data (the observations $x_1, \dots, x_{200}$), and we will analyse this vector. We construct a 95% two-sided confidence interval.

```
mu.hat<-mean(x)
error <- qnorm(0.975)*1/sqrt(n)
left <- mu.hat-error
right <- mu.hat+error
```

Then the interval $(left, right)$ is a 95% two-sided confidence interval for μ .

To check, and illustrate, if the theoretical covering probability 0.95 is correct we will do many times (N times) the above, so we will construct N confidence intervals, and see how many times the corresponding constructed confidence interval $(left, right)$ contains the correct value μ . In the code below the value of N is chosen as 1000, you may choose another (large) value if you prefer.

```

N<-1000
count<-0
for (i in (1:N)){
  x<-rnorm(n,mean=mu,sd=1)
  mu.hat<-mean(x)
  error <- qnorm(0.975)*1/sqrt(n)
  left <- mu.hat-error
  right <- mu.hat+error
  count<-count+as.double(left<=mu & mu<=right)
}

```

Then the variable *count* contains the number of times the interval contains the correct value of μ , and

count/N

is the proportion.

- (a) Does it seem as the covering proportion 0.95 is correct?
- (b) Use the law of large numbers to motivate what count/N converges to, when some number (which?) converges to infinity.

Note that this is typically not possible to do for real data situations: We can construct $N = 1000$ confidence intervals since we have simulated data. In real life we have *one single* data sample that we need to analyse.

2. We want to construct a confidence interval for the variance in a $N(\mu, \sigma^2)$ -distribution. We generate 300 data points from a $N(\mu, \sigma^2)$ -distribution.

```

n<-300
mu<-2
sigma<-2
x<-rnorm(n,mean=mu,sd=sigma)

```

The vector x now contains the data (the observations $x_1, \dots, x_{300}$), and we will analyse this vector. We construct a 95% two-sided confidence interval.

```

sigma.hat<-sd(x)
left <- (n-1)*sigma.hat^2/qchisq(0.975,df=n-1)
right <- (n-1)*sigma.hat^2/qchisq(0.025,df=n-1)

```

Then the interval $(\text{left}, \text{right})$ is a 95% two-sided confidence interval for σ^2 . To check if the theoretical covering probability is 0.95, we will construct N confidence intervals, and see how many times the corresponding constructed confidence interval $(\text{left}, \text{right})$ contains the correct value σ^2 .

```

N<-1000
count<-0
for (i in (1:N)){
  x<-rnorm(n,mean=mu,sd=sigma)
  sigma2.hat<-sd(x)^2
  left <- (n-1)*sigma2.hat/qchisq(0.975,df=n-1)
  right <- (n-1)*sigma2.hat/qchisq(0.025,df=n-1)
  count<-count+as.double(left<= sigma^2 & sigma^2<=right)
}

```

The variable *count* contains the number of times the interval contains the correct value of σ^2 , and

`count/N`

is the proportion of times it does.

- (a) Does it seem as the covering proportion 0.95 is correct?
- (b) Use the law of large numbers to motivate what *count/N* converges to, when some number (which?) converges to infinity.

3 Hypothesis testing

This part of the computer lab consists of constructing tests for simple and composite null hypotheses, to illustrate the significance level and determine and plot the power function.

1. We test the hypotheses

$$\begin{aligned} H_0 : & \quad \theta = 0, \\ H_1 : & \quad \theta \neq 0, \end{aligned}$$

in a $N(\theta, 1)$ distribution. We generate a sample of 200 data points from the $N(0, 1)$ distribution.

```
n<-200
mu<-0
n<-200
x<-rnorm(n,mean=mu,sd=1)
```

The test statistic and the tests result are, for a test on level 0.05,

```
teta.hat<-mean(x)
test.stat<-teta.hat/(1/sqrt(n))
reject<-(abs(test.stat)>qnorm(0.975))
```

Then *reject* is a logical variable taking the value *TRUE* or *FALSE* depending on the result of the test. We illustrate the significance level by counting the number of times that we reject the null hypothesis when we do $N = 1000$ tests.

```
N<-1000
count<-0
for (i in (1:N)){
  x<-rnorm(n,mean=mu,sd=1)
  teta.hat<-mean(x)
  test.stat<-teta.hat/(1/sqrt(n))
  teta.hat<-mean(x)
  test.stat<-teta.hat/(1/sqrt(n))
  reject<-(abs(test.stat)>qnorm(0.975))
  count<-count+as.double(reject)
}
count/N
```

- (a) Does a significance value of 0.05 seem to be correct?
- (b) Use the LLN to motivate what *count/N* converges to, when some number (which?) converges to infinity.

We next will plot the power function $\pi(\theta)$ for the above test, on the interval $(-2, 2)$. Recall the derivation of the power function for a two-sided test of θ in $N(\theta, 1)$, in the lecture notes.

- (a) We first use $n = 200$ as above

```
x.points<-(1:1000)/250-2
power<-1-(pnorm(qnorm(0.975)-x.points/(1/sqrt(n)))
          -pnorm(qnorm(0.025)-x.points/(1/sqrt(n))))
plot(x.points,power,type="l")
```

- (b) How does the power of the test change when you have fewer data points, for instance $n = 25$?

```
n<-25
x.points<-(1:1000)/250-2
power<-1-(pnorm(qnorm(0.975)-x.points/(1/sqrt(n)))
          -pnorm(qnorm(0.025)-x.points/(1/sqrt(n))))
plot(x.points,power,type="l")
```

- (c) What happens when you have many data points, say $n = 1000$?

```
n<-1000
x.points<-(1:1000)/250-2
power<-1-(pnorm(qnorm(0.975)-x.points/(1/sqrt(n)))
          -pnorm(qnorm(0.025)-x.points/(1/sqrt(n))))
plot(x.points,power,type="l")
```

Explain your results.